

Hotel Management Backend

Komplet Eksamensguide — Databases for Developers

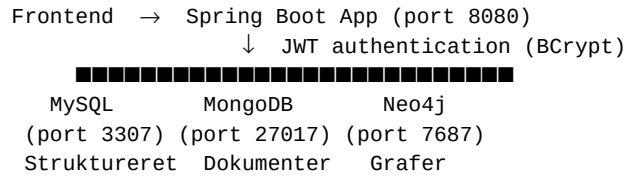
Torsdag — Eksamen

Indhold

- DEL 1 — Systemarkitektur
- DEL 2 — MySQL: Relational Database Design
- DEL 3 — Normalisering: 1NF, 2NF, 3NF
- DEL 4 — MongoDB Design
- DEL 5 — Neo4j Graph Model
- DEL 6 — Cross-Database Query
- DEL 7 — Transaktioner og ACID
- DEL 8 — Indexes og Performance
- DEL 9 — Security og Auditing
- DEL 10 — Stored Procedures, Views og Events
- DEL 11 — Trade-offs og Refleksioner
- DEL 12 — Fagtermer Quick Reference
- DEL 13 — Mundtlige Spørgsmål og Svar

DEL 1 — Systemarkitektur

Dit system er en **Hotel Management Backend** bygget med Spring Boot 4. Applikationen bruger tre databaser parallelt til at løse tre forskellige problemer:



- **MySQL** — kilden til sandheden. Strukturerede, relationelle data med ACID-garantier. Bruges til reservationer, gæster, rum, priser og billing.
- **MongoDB** — kopi med embedded dokumenter. En gæsts fulde historik i ét kald. Bruges også til AI-interaktions-logs (ustrukturerede LLM-samtaler).
- **Neo4j** — relation-grafen. Beskriver forbindelser: hvem bookede hvad, hvilke cleaners er assigned til hvilke rum.

Migration: Data starter i MySQL og kopieres manuelt til MongoDB og Neo4j via POST /api/migrate. I produktion ville man bruge event-driven sync.

DEL 2 — MySQL: Relational Database Design

Hvad er en relationel database?

En relationel database gemmer data i **tabeller** med **rækker** og **kolonner**. Tabeller relaterer til hinanden via nøgler. Databasemotoren (InnoDB i jeres tilfælde) håndhæver regler automatisk.

Primary Key (PK) — Primær nøgle

En **primær nøgle** identificerer unikt én enkelt række. Ingen to rækker har samme PK. AUTO_INCREMENT lader databasen selv tælle (1, 2, 3...).

```
guest_id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY
```

Foreign Key (FK) — Fremmed nøgle

En **fremmed nøgle** peger på en PK i en anden tabel og skaber relationen. Databasen afviser operationer der bryder referencen.

```
-- I reservation-tabellen:  
FOREIGN KEY (guest_id) REFERENCES guest(guest_id) ON DELETE CASCADE
```

ON DELETE CASCADE: Slettes gæsten, slettes alle gæstens reservationer automatisk.

ON DELETE SET NULL: Slettes gæsten, sættes FK til NULL.

ON DELETE RESTRICT: Kan ikke slette gæsten mens der er reservationer.

Kardinalitet

Kardinalitet beskriver *hvor mange* af den ene type der relaterer til *hvor mange* af den anden:

Type	Beskrivelse	Eksempel i jeres system
1:1	Én til én	reservation ↔ bill (én bill per reservation, UNIQUE FK)
1:N	Én til mange	room_type → mange rooms
M:N	Mange til mange	reservation ↔ guest (via reservation_guest junction)

Constraints — Begrænsninger

Constraint	Hvad det gør	Eksempel
PRIMARY KEY	Unik + NOT NULL	guest_id PRIMARY KEY
UNIQUE	Ingen duplikater tilladt	email VARCHAR UNIQUE
NOT NULL	Feltet må ikke være tomt	check_in_date NOT NULL
FOREIGN KEY	Refereret række skal eksistere	room_type_id FK→room_type
ENUM	Kun specificerede værdier	role ENUM('ADMIN','STAFF','CLEANER')
DEFAULT	Standardværdi	occupied BIT DEFAULT 0

Jeres 13 tabeller

Tabel	PK	Vigtige FK'er	Formål
user_account	id (BIGINT)	—	Login, roller
guest	guest_id (BIGINT)	—	Gæstinfo
room_type	room_type_id (INT)	—	Single/Double/Suite
season_rate	rate_id (INT)	room_type_id	Priser per sæson
room	room_id (INT)	room_type_id	Individuelle rum
reservation	reservation_id (INT)	guest_id, room_id, room_type_id, booked	Bookinger
reservation_guest	(reservation_id, guest_id)	ingen FK	M:N junction
bill	bill_id (INT)	reservation_id (UNIQUE)	1:1 med reservation
bill_item	bill_item_id (INT)	bill_id	Regningslinjer
cleaner	cleaner_id (INT)	—	Rengøringspersonale
room_cleaning_task	task_id (INT)	room_id	Rengøringsopgaver
room_cleaning_assignment	(task_id, cleaner_id)	begge FK	M:N junction
extra_service	extra_service_id (INT)	—	Ekstraydelser

DEL 3 — Normalisering: 1NF, 2NF, 3NF

Normalisering er processen med at organisere din database for at undgå:

- **Redundans** — samme data gemt flere steder (f.eks. rumtype-navn gentaget i alle rum)
- **Update anomaly** — opdaterer ét sted, glemmer et andet
- **Insertion anomaly** — kan ikke tilføje fornuftigt data
- **Deletion anomaly** — sletter én ting, mister utilsigtet andet

1NF — First Normal Form

Regel: Hvert felt indeholder kun én **atomisk** (udelelig) værdi. Ingen lister, ingen arrays, ingen komma-separerede værdier i ét felt.

IKKE 1NF (dårligt design):

```
guest_id | name   | rooms_booked
1        | Magnus | "101, 102, 105" ← FORKERT: liste i ét felt
```

Jeres design (1NF overholdes): Reservationer er i sin *egen* tabel med én reservation per række. credit_card_last4 gemmer kun de 4 sidste cifre — en atomisk værdi, ikke hele kortnummeret.

2NF — Second Normal Form

Regel: Forudsætter 1NF, *plus*: alle ikke-nøgle-felter skal afhænge af **hele** primærnøglen. Relevant når du har **composite primary keys**.

Eksempel fra jeres junction-tabel:

```
-- reservation_guest har composite PK:
PK = (reservation_id, guest_id)
is_primary ← afhænger af BEGGE kolonner ✓

-- IKKE 2NF (dårligt):
guest_email ← afhænger KUN af guest_id, ikke reservation_id ✗
```

Løsning: guest_email hører til i guest-tabellen.

3NF — Third Normal Form

Regel: Forudsætter 2NF, *plus*: ingen **transitive afhængigheder**. Det vil sige: ikke-nøgle-felter må ikke afhænge af *andre* ikke-nøgle-felter.

En **transitiv afhængighed** er: $A \rightarrow B \rightarrow C$, hvor C afhænger af B som afhænger af PK (A). C burde flyttes til sin egen tabel.

Dårligt design (IKKE 3NF):

```
reservation(reservation_id PK, room_type_id, room_type_name, max_occupancy)
```

Her: room_type_name og max_occupancy afhænger af room_type_id, ikke af reservation_id direkte. Det er en transitiv afhængighed.

Jeres faktiske 3NF-design:

```
reservation(reservation_id PK, room_type_id FK, ...)
room_type (room_type_id PK, name, max_occupancy) ← eget bord
```

Snapshot-undtagelse (vigtig at forklare!): booked_nightly_price gemmes direkte på reservation selvom prisen også findes i season_rate. Det er en *bevidst designbeslutning*: prisen på bookingtidspunktet skal bevares, selvom sæsonprisen ændrer sig næste år. Det er et **historisk snapshot**, ikke en normaliseringsovertrædelse.

DEL 4 — MongoDB Design

Hvad er MongoDB?

MongoDB er en **dokumentdatabase** (NoSQL). I stedet for tabeller med rækker gemmer den **dokumenter** (JSON-lignende) i **collections**. Der er intet fast schema — hvert dokument kan have forskellige felter.

MySQL	MongoDB
Database	Database
Tabel	Collection
Række	Document
Kolonne	Field
JOIN	\$lookup (eller embedding)
Schema required	Schema-free (fleksibelt)

Embedding vs. Referencing

Det centrale designvalg i MongoDB: skal relaterede data **embeddes** (kopieres ind i dokumentet) eller **refereres** (gemmes separat med et ID)?

Embedded — gæst med reservationer (jeres valg)

```
{
  "_id": "...",
  "guestId": 5,
  "firstName": "Magnus",
  "email": "magnus@hotel.dk",
  "reservations": [           ← EMBEDDED i guest-dokumentet
    {
      "reservationId": 42,
      "referenceNo": "RES-2024-001",
      "checkInDate": "2024-06-01",
      "roomType": "Double",
      "bills": [               ← EMBEDDED i reservation
        {
          "billId": 1,
          "totalAmount": 2400.00,
          "items": [
            { "itemType": "ROOM", "description": "2 nætter", "unitPrice": 1200 }
          ]
        }
      ]
    }
  ]
}
```

Fordel: "Hent alt om gæst Magnus" = ét enkelt database-kald, ingen joins.

Ulempe: Opdateres en reservation, skal den opdateres to steder.

Referenced — separate collections

rooms, cleaners, season_rates er *separate* collections med references via ID. Disse embeddes *ikke* i guest-dokumentet fordi:

- Et room eksisterer uafhængigt af reservationer
- Mange reservationer deler samme room_type — embedding ville skabe massiv redundans
- Rooms opdateres hyppigt (status, clean_status)

ai_interactions — kun MongoDB

```
{
  "sessionId": "abc123",
  "userMessage": "Hvad koster et dobbeltværelse i juli?",
  "aiResponse": "Et dobbeltværelse koster...",
  "model": "llama3",
  "tokensUsed": 245,
  "timestamp": "2024-06-01T14:30:00"
}
```

LLM-samtaler gemmes *kun* i MongoDB — de har ingen fast struktur, og der er ingen relationer der kræver ACID.

DEL 5 — Neo4j Graph Model

Terminologi

Term	Beskrivelse	Eksempel
Node	Et objekt i grafen	(:Guest), (:Room), (:Reservation)
Relationship	Forbindelsen mellem to noder	[:BOOKED_BY], [:ASSIGNED_TO]
Label	Typen af en node	:Guest, :Room, :Bill
Property	Egenskab på node eller relation	firstName, roomNumber, checkIn
Direction	Relationer har retning (pil)	Guest → BOOKED_BY → Reservation
Traversal	At navigere langs piler i grafen	Hop fra guest → reservation → room
Cypher	Neo4js query-sprog	MATCH (g:Guest)-[:BOOKED_BY]->(r:Reservation)

Jeres graph model

```
(:Guest {guestId, firstName, lastName, email})
  -[:BOOKED_BY]->
(:Reservation {reservationId, referenceNo, status, checkIn, checkOut, nights})
  -[:ASSIGNED_TO]->
(:Room {roomId, roomNumber, roomStatus, cleanStatus, occupied})
  -[:IS_TYPE]->
(:RoomType {name, maxOccupancy})
  -[:HAS_RATE]->
(:SeasonRate {season, pricePerNight})

(:Reservation)-[:HAS_BILL]->(:Bill)-[:CONTAINS_ITEM]->(:BillItem)

(:Room)-[:HAS_CLEANING_TASK]->(:RoomCleaningTask)
(:Cleaner)-[:ASSIGNED_TO]->(:RoomCleaningTask)
```

Hvorfor Neo4j frem for SQL til visse queries?

Spørgsmål: "Hvilke cleaners rengør rum der er booket af gæster med 3+ reservationer?"

```
-- SQL: kræver 4 JOINS + subquery
SELECT DISTINCT c.first_name, c.last_name
FROM cleaner c
JOIN room_cleaning_assignment rca ON c.cleaner_id = rca.cleaner_id
JOIN room_cleaning_task rct ON rca.task_id = rct.task_id
JOIN room r ON rct.room_id = r.room_id
JOIN reservation res ON r.room_id = res.room_id
WHERE res.guest_id IN (
  SELECT guest_id FROM reservation GROUP BY guest_id HAVING COUNT(*) >= 3
);

-- Cypher: naturlig traversal langs piler
MATCH (g:Guest)-[:BOOKED_BY]->(res:Reservation)-[:ASSIGNED_TO]->(r:Room)
  -[:HAS_CLEANING_TASK]->(task:RoomCleaningTask)
  <-[:ASSIGNED_TO]-(c:Cleaner)
WHERE size((g)-[:BOOKED_BY]->()) >= 3
RETURN DISTINCT c.firstName, c.lastName
```

DEL 6 — Cross-Database Query

Scenario: Find alle CONFIRMED reservationer for rum 101

MySQL

```
SELECT g.first_name, g.last_name, r.reference_no,
       r.check_in_date, r.check_out_date
FROM reservation r
JOIN guest g ON r.guest_id = g.guest_id
JOIN room rm ON r.room_id = rm.room_id
WHERE rm.room_number = '101'
      AND r.status = 'CONFIRMED';
```

JOIN kombinerer rækker fra to tabeller baseret på et fælles felt. **WHERE** filtrerer resultater.

MongoDB

```
db.guests.aggregate([
  { $unwind: "$reservations" },      // fold reservations-array ud
  { $match: {
    "reservations.assignedRoomNumber": "101",
    "reservations.status": "CONFIRMED"
  }},
  { $project: {
    firstName: 1, lastName: 1,
    "reservations.referenceNo": 1,
    "reservations.checkInDate": 1
  }}
])
```

\$unwind: "folder" et embedded array ud til individuelle dokumenter.

\$match: filtrerer (som WHERE).

\$project: vælger felter (som SELECT).

aggregate: en pipeline af operationer der kører i sekvens.

Neo4j (Cypher)

```
MATCH (g:Guest)-[:BOOKED_BY]->(res:Reservation)-[:ASSIGNED_TO]->(r:Room)
WHERE r.roomNumber = '101' AND res.status = 'CONFIRMED'
RETURN g.firstName, g.lastName, res.referenceNo, res.checkIn
```

MATCH: definerer mønsteret du leder efter i grafen.

WHERE: filtrerer på properties.

RETURN: specificerer hvad der returneres.

Sammenligning

	MySQL	MongoDB	Neo4j
Syntaks	Tabelbaseret, explicit JOIN	Pipeline-orienteret	Mønsterilpasning
Bedst til	Strukturerede relationer	Hierarkiske dokumenter	Mange-hop traversals
Skalering	Vertikal	Horisontal (sharding)	Horisontal (clustering)
ACID	Fuld	Dokument-niveau	Fuld (enterprise)

Query-sprog	SQL	MQL / aggregation	Cypher
-------------	-----	-------------------	--------

DEL 7 — Transaktioner og ACID

Hvad er en transaktion?

En **transaktion** er en gruppe af database-operationer der enten **alle lykkes** eller **alle fejler** — aldrig halvt. Det er som en atomisk handling der ikke kan afbrydes midt i.

Konkret eksempel: Gæst tjekker ind

Disse tre operationer skal alle lykkes eller alle rulles tilbage:

```
BEGIN; -- start transaktion

UPDATE reservation
SET status = 'CHECKED_IN'
WHERE reservation_id = 42;           -- trin 1

UPDATE room
SET occupied = 1, room_status = 'OCCUPIED'
WHERE room_id = 15;                 -- trin 2

INSERT INTO bill (reservation_id, opened_at, total_amount)
VALUES (42, NOW(), 0.00);           -- trin 3 – åbn regning

COMMIT; -- gem alt permanent

-- Hvis noget fejler:
ROLLBACK; -- fortryd ALLE tre operationer
```

I Spring Boot sker dette automatisk med `@Transactional` på service-klassen.

ACID — de fire garantier

Bogstav	Navn	Betyder	Eksempel
A	Atomicity	Enten alt eller intet	Check-in: alle 3 steps lykkes eller ingen
C	Consistency	Fra valid til valid tilstand	FK-constraints valideres — kan ikke booke ikke-eksisterende rum
I	Isolation	Concurrent transaktioner ser ikke hinandens data	En gæst kan booke et rum — kun én lykkes
D	Durability	Committed data forbliver gemt	Server crasher efter COMMIT — data er stadig der

DEL 8 — Indexes og Performance

Hvad er et index?

Et **index** er en separat datastruktur der gør søgning hurtigere. Tænk på det som en bogs indeks bagerst: i stedet for at læse alle sider slår du direkte op og får sidenummeret.

- **Uden index:** Full table scan — databasen gennemgår alle rækker én efter én → langsomt
- **Med index:** Springer direkte til rækken → $O(\log n)$ i stedet for $O(n)$

B-tree index

Den mest almindelige index-type. Balanced Tree — sorteret hierarkisk struktur. God til:

- Eksakt match: `WHERE email = 'magnus@hotel.dk'`
- Range queries: `WHERE check_in_date BETWEEN '2024-01-01' AND '2024-12-31'`
- Sortering: `ORDER BY last_name`

Composite index

Et **composite index** dækker *flere kolonner*. Rækkefølgen af kolonner har betydning — søg altid på den første kolonne.

```
INDEX idx_reservation_dates (check_in_date, check_out_date)
-- Dækker: WHERE check_in_date = X (effektivt)
-- Dækker: WHERE check_in_date = X AND check_out_date = Y (effektivt)
-- Dækker IKKE: WHERE check_out_date = Y alene (ineffektivt)
```

Jeres indexes fra 01_database_create.sql

Tabel	Index	Type	Begrundelse
guest	idx_guest_email	B-tree	Login-lookup — hyppigt brugt
guest	idx_guest_name (last_name, first_name)	Composite	Søgning på navn
reservation	idx_reservation_dates	Composite	Dato-range queries (ledige rum)
reservation	idx_reservation_status	B-tree	Filter: alle CONFIRMED
reservation	idx_reservation_guest	B-tree	FK join: gæstens reservationer
bill	idx_bill_reservation	B-tree	FK join performance
room	idx_room_status + idx_room_clean_status	B-tree	Filtrer ledige/beskidte rum
audit_log	idx_audit_table/operation/timestamp	B-tree	Hurtig audit-søgning

Hvad I IKKE indexed: `bill_item.description` — tekstsøgning bruges ikke, og et index ville sænke inserts unødigt. `occupied BIT` — kun 2 værdier (lav cardinality), databasen henter alligevel ~50% af rækkerne.

DEL 9 — Security og Auditing

JWT — JSON Web Token

JWT er en måde at bevise identitet uden at serveren skal huske hvem du er (**stateless**). En JWT består af tre dele separeret af punktum:

- **Header** — algoritme brugt til signering (HS512 i jeres kode)
- **Payload** — brugers data: username, rolle, udløbstid
- **Signature** — kryptografisk signatur der beviser tokenet ikke er manipuleret

```
// Flow:
1. POST /api/auth/login { username: "admin", password: "admin123" }
2. Server tjekker BCrypt-hash mod user_account-tabellen
3. Server returnerer: { token: "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJhZG1pbi..." }
4. Klienten sender: Authorization: Bearer eyJhbGciOiJIUzUxMiJ9...
5. Server verificerer signatur – ingen database-kald nødvendigt!
```

BCrypt — Password Hashing

Passwords gemmes aldrig i plain text. BCrypt er en one-way hash-funktion:

```
"admin123" → BCrypt → "$2a$10$N9qo8uL0ickgx2ZMRZoMye..."
```

One-way: Kan ikke gå baglæns fra hash til password.

Salt: BCrypt tilføjer automatisk random data til hvert password — to brugere med samme password får forskellige hashes.

Roller og adgangskontrol

Rolle	Adgang
ADMIN	Fuld adgang — kan slette, oprette, se alt
STAFF	Kan se og opdatere reservationer, men ikke slette
CLEANER	Kan kun se og opdatere cleaning tasks

SQL Injection prevention

SQL Injection er et angreb hvor ondsindet input manipulerer SQL-queries:

```
// FARLIGHT – brug aldrig string-konkatenering:
String q = "SELECT * FROM guest WHERE email = '" + email + "'";
// Angreb: email = "' OR '1'='1" → returnerer ALLE gæster!

// SIKKERT – parameterized query via Spring Data JPA:
Optional<Guest> findByEmail(String email);
// Genererer: SELECT * FROM guest WHERE email = ?
// email-værdien sendes separat – behandles som DATA, ikke kode
```

Auditing via Triggers (05_audit.sql)

En **trigger** er SQL-kode der automatisk kører ved INSERT/UPDATE/DELETE — applikationen behøver ikke aktivt at gøre noget.

audit_log tabellen logger ændringer på: reservation, bill, bill_item, guest, room.

Hvert audit-entry gemmer:

Felt	Indhold
table_name	Hvilken tabel der ændrede sig
operation_type	INSERT / UPDATE / DELETE
record_id	ID på den ændrede record
old_values (JSON)	Hvad stod der FØR ændringen
new_values (JSON)	Hvad er der NU
changed_by	MySQL-brugeren (CURRENT_USER())
changed_at	Tidsstempel for ændringen

DEL 10 — Stored Procedures, Views og Events

Stored Procedure — sp_CalculateFinalBill

En **stored procedure** er foruddefineret SQL gemt i databasen der kan kaldes med parametre — som en funktion i programmering. Logikken er i databasen, ikke i applikationskoden.

```
CALL sp_CalculateFinalBill(42);
-- Returnerer: { final_bill_dkk: 3600.00 }
```

Proceduren: (1) henter nætter og nattepris fra reservation, (2) beregner rumtotal = nætter × pris, (3) summer ekstra services fra bill_item, (4) opdaterer bill.total_amount, (5) returnerer beløbet.

Stored Function — fn_GetRoomRate

En **stored function** returnerer én enkelt værdi og kan bruges direkte i SQL:

```
SELECT fn_GetRoomRate(2, 'Summer') AS daily_rate;
-- Returnerer: 1200.00
```

Views — virtuelle tabeller

Et **view** er en gemt SQL-query der opfører sig som en tabel. Det gemmer *ikke* data — beregner resultatet live.

View	Viser
vw_HousekeepingList	Alle beskidte/inspekterede rum med rum-nr og type
vw_ReservationDetails	Reservationer med gæst-info, rum-nr og estimeret pris
vw_BillDetails	Bills med gæst-navn, åbnet/lukket, total og antal linjer
vw_recent_audit_logs	Seneste 100 audit-entries med formaterede tidsstempler

Events — automatisk kørende SQL

Events er som cron jobs i databasen — SQL der kører på et tidsplan:

Event	Kører	Gør
ev_AutoCancelStaleReservations	Dagligt kl. 02:00	Annulerer PENDING-reservationer ældre end 7 dage — triggerer også tr_RoomS
ev_MonthlyCloseOldBills	1. i måneden kl. 03:00	Lukker bills der er åbne 30+ dage efter CHECKED_OUT

Triggers i jeres system

Trigger	Hvornår	Hvad den gør
tr_AfterCheckout	AFTER UPDATE reservation	Sætter rum til 'Vacant'/'Dirty' ved checkout
tr_RoomStatusUpdate	AFTER UPDATE reservation	Sætter rum OCCUPIED=1 ved CONFIRMED, AVAILABLE=0 ved CANCELLE
tr_audit_*	AFTER INSERT/UPDATE/DELETE	Lagrer alle ændringer til audit_log

DEL 11 — Trade-offs og Refleksioner

Hvorfor MySQL til hotel-core data?

- Reservationsdata er **struktureret og relationelt** — gæst booker rum til en pris, alt hænger logisk sammen
- **ACID-garantier** er kritiske: du kan ikke have en betaling uden en reservation
- Velkendt schema der sjældent ændrer sig
- Komplekse **JOIN-queries** er naturlige (vw_ReservationDetails bruger 4 tabeller)

Hvorfor MongoDB?

- En gæsts **fulde historik** i ét dokument-kald — ingen joins nødvendige
- ai_interactions er helt ustrukturerede LLM-logs — passer ikke i et fast schema
- **Fleksibelt** for fremtidige felter uden schema-migration (bare tilføj et nyt felt)
- God til read-heavy workloads med nested/hierarkisk data

Hvorfor Neo4j?

- Komplekse **relationsnavigation** er trivielt (følg pilene) — ville kræve mange JOINS i SQL
- Visualisering af hotellets forbindelser giver operationelt overblik
- Kan udvides til fx **anbefalinger**: "gæster der bookede dette rum bookede også..."
- Graf-traversals skalerer bedre end rekursive SQL-queries ved dybe relationer

Hvad I ville gøre anderledes

- **Dato-typer i Neo4j**: Datoer gemtes som strings ("2024-06-01"). Neo4j har native date og datetime-typer der er mere effektive og kan bruges i Cypher-beregninger. Opdaget undervejs i projektet.
- **Sync-strategi**: Data migreres manuelt via POST /api/migrate. I produktion ville man bruge **outbox pattern** eller **Change Data Capture (CDC)** med Kafka, så ændringer propageres automatisk.
- **Multi-store Spring Boot**: Kræver eksplicit konfiguration af to transaction managers (JpaTransactionManager + Neo4jTransactionManager). Spring Boot 4's auto-configuration håndterer ikke dette automatisk — lærte dette undervejs.

DEL 12 — Fagtermer Quick Reference

Term	Forklaring
Primary Key (PK)	Unikt ID for en tabel-række — ingen duplikater
Foreign Key (FK)	Reference til en anden tabels PK — skaber relationen
Kardinalitet	1:1, 1:N, M:N — 'hvor mange til hvor mange'
Junction table	Mellemtabel for M:N relationer (reservation_guest)
Composite PK	Primærnøgle bestående af to+ kolonner
Constraint	Regel databasen håndhæver automatisk (NOT NULL, UNIQUE, FK)
1NF	Atomare felter — ingen lister i én kolonne
2NF	Alle felter afhænger af hele PK (relevant ved composite PK)
3NF	Ingen transitive afhængigheder ($A \rightarrow B \rightarrow C$ flyttes til eget bord)
Transitiv afhængighed	Ikke-nøgle-felt afhænger af andet ikke-nøgle-felt
Snapshot	Historisk kopi af data på et givent tidspunkt (booked_nightly_price)
Redundans	Samme data gemt unødvendigt flere steder
NoSQL	Ikke-relationel database — MongoDB, Neo4j
Document	JSON-lignende record i MongoDB
Collection	Gruppe af dokumenter i MongoDB (svarende til tabel)
Embedding	Relaterede data kopieres ind i samme dokument
Referencing	ID-reference til et separat dokument
Aggregation pipeline	MongoDB: sekvens af operationer (\$unwind, \$match, \$project)
Node	Et objekt i en grafmodel
Relationship	Forbindelse/kant mellem to noder i Neo4j
Label	Typen af en node (:Guest, :Room)
Property	Egenskab på node eller relation (firstName, checkIn)
Traversal	At navigere langs kanter i en graf
Cypher	Neo4js query-sprog (MATCH, WHERE, RETURN)
Transaktion	Gruppe af operationer — alle lykkes eller alle fejler
ACID	Atomicity, Consistency, Isolation, Durability
Atomicity	Enten alt eller intet
Consistency	Fra valid til valid tilstand
Isolation	Concurrent transaktioner ser ikke hinandens uncommitted data
Durability	Committed data forbliver gemt ved crash
COMMIT	Gem alle transaktionens operationer permanent
ROLLBACK	Fortryd alle operationer i transaktionen
Index	Datastruktur der gør søgning hurtigere

B-tree index	Standard tree-baseret index — god til range-queries og exact match
Composite index	Index der dækker flere kolonner
Full table scan	Databasen gennemgår alle rækker — langsomt uden index
Cardinality	Antal unikke værdier i en kolonne
JWT	JSON Web Token — token-baseret authentication
BCrypt	One-way password hashing algoritme
Salt	Random data tilføjet password før hashing — forhindrer rainbow tables
SQL Injection	Angreb via ondsindet SQL-kode i inputfelter
Parameterized query	Sikker SQL med ? placeholders — input behandles som data
Trigger	SQL der automatisk kører ved INSERT/UPDATE/DELETE
Stored procedure	Gemt SQL-kode kaldt med parametre (CALL sp_...)
Stored function	Som procedure, returnerer én værdi — bruges i SQL
View	Gemt query der opfører sig som virtuel tabel
Event	SQL der kører automatisk på tidsplan (cron job i databasen)
Sharding	Horisontal skalering — data spredt på flere servere
CDC / Outbox pattern	Event-driven sync mellem databaser i produktion
InnoDB	MySQL's storage engine — understøtter ACID og FK

DEL 13 — Mundtlige Spørgsmål og Svar

De spørgsmål der stilles oftest ved mundtlig database-eksamen:

Spørgsmål: Hvorfor bruger I tre databaser i stedet for bare MySQL?

Svar: MySQL giver ACID-garantier for kernedata der er relationelt struktureret. MongoDB giver os mulighed for at hente en gæsts fulde historik — alle reservationer, bills og items — i ét kald uden joins, fordi vi embedder data. Neo4j giver os naturlig graph-traversal for spørgsmål om forbindelser, f.eks. hvilke cleaners arbejder på rum booket af specifikke gæster. Hvert database-valg løser et problem de andre ikke er bedst til.

Spørgsmål: Er din reservation-tabel i 3NF?

Svar: Ja. Alle attributter afhænger direkte af reservation_id. booked_nightly_price ser ud som en transitiv afhængighed af season_rate, men det er et bevidst snapshot — vi gemmer prisen på bookingtidspunktet fordi sæsonpriser kan ændre sig. Det er ikke en normaliseringsovertrædelse, det er en designbeslutning der sikrer historisk korrekthed.

Spørgsmål: Hvad er forskellen på embedding og referencing i MongoDB?

Svar: Embedding betyder at relaterede data kopieres direkte ind i dokumentet — vi har reservationer embedded i gæster fordi vi næsten altid henter dem sammen. Referencing betyder at vi gemmer et ID til et separat dokument — vi referencer rooms og cleaners fordi de eksisterer uafhængigt og deles af mange.

Spørgsmål: Hvad sker der hvis MongoDB og MySQL kommer ud af sync?

Svar: I vores nuværende system migreres data manuelt via POST /api/migrate. Hvis MySQL opdateres uden at køre migration, vil MongoDB have forældet data. I en produktionsløsning ville vi bruge Change Data Capture eller et outbox pattern med en message broker som Kafka, så ændringer propageres automatisk og pålideligt.

Spørgsmål: Vis mig SQL injection prevention i din kode

Svar: Vi bruger Spring Data JPA som automatisk genererer parameterized queries. I stedet for at konkatenerer bruger-input direkte i SQL-strings, bruges method-derived queries som findByEmail(String email), hvilket genererer 'SELECT * FROM guest WHERE email = ?' hvor databasen behandler input som data, aldrig som kode.

Spørgsmål: Hvorfor har reservation to FK'er til room?

Svar: assigned_room_id er det rum der formelt er tildelt på bookingtidspunktet, room_id er det rum der faktisk bruges ved check-in. De kan afvige ved f.eks. rum-opgraderinger eller reparationer.

Spørgsmål: Hvad er formålet med audit_log tabellen?

Svar: Audit-tabellen logger alle INSERT, UPDATE og DELETE på kritiske tabeller via database-triggers. Hvert entry gemmer hvad der stod FØR og EFTER ændringen som JSON, hvem der lavede ændringen, og hvornår. Det bruges til compliance, fejlsøgning, og sikkerhed — f.eks. 'hvem ændrede reservationens status og hvornår?'

Spørgsmål: Hvad er et composite index og hvornår bruger I det?

Svar: Et composite index dækker flere kolonner i én datastruktur. Vi bruger idx_reservation_dates(check_in_date, check_out_date) fordi vi ofte søger på begge datoer i availability-queries. Rækkefølgen betyder noget — indexet dækker queries der starter med check_in_date.

Spørgsmål: Hvad er forskellen på en stored procedure og en trigger?

Svar: En stored procedure kaldes eksplicit med parametre, f.eks. CALL sp_CalculateFinalBill(42). En trigger kører automatisk som reaktion på en database-hændelse som INSERT eller UPDATE — ingen eksplicit kald nødvendig. tr_AfterCheckout kører fx automatisk når reservation.status ændres til 'Checked Out'.

Spørgsmål: Hvad er ACID og hvorfor er det vigtigt for et hotelsystem?

Svar: ACID er fire garantier for database-transaktioner. Atomicity: check-in-operationen (opdater reservation + opdater rum + opret bill) lykkes komplet eller slet ikke. Consistency: FK-constraints sikrer at vi ikke kan have en reservation til et ikke-eksisterende rum. Isolation: to gæster der booker det samme rum samtidig ser ikke hinandens uncommitted data. Durability: når en betaling er committed forbliver den gemt selv ved server-nedbrud.

Held og lykke til eksamen torsdag!